

Agent Design Sheet

Restaurant Booking Assistant

Week 17B · Design Studio · a worked example in a different domain

THE EXAMPLE

Bistro Uno Takes Table Bookings Over Chat

Bistro Uno is a fictional 60-seat restaurant. Its chat assistant answers booking requests: it collects the date, time and party size, checks free tables, and books.

Groups of more than 8 pay a ₱2,000 deposit, **and a manager must approve them.**

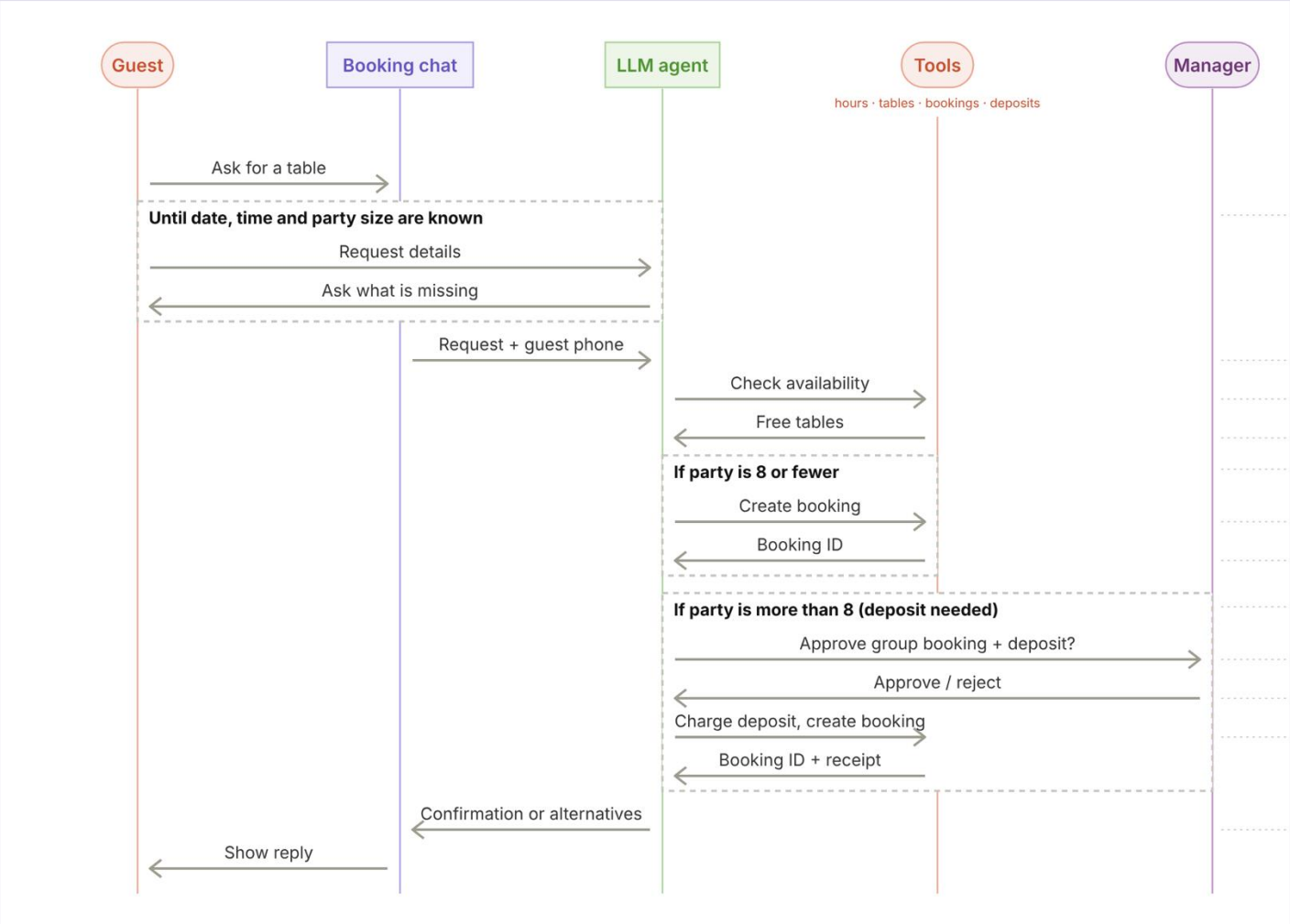
1. **What it does alone**
   Answers opening hours, checks availability, books parties of 8 or fewer, cancels a guest's own booking.

2. **Where a person steps in**
   Group bookings and deposits: the manager approves before any money moves.

3. **What it never does**
   Refund deposits, change prices, or see other guests' details.

PART 1 · SEQUENCE DIAGRAM

One Booking, End to End



What the Agent Writes Down, and What It Reads

```
// scratchpad/chat-0457.json
// written after every step
{
  "chat_id": "chat-0457",
  "guest_name": "Rina",
  "phone": "+63 900 000 0457",
  "date": "2026-11-07",
  "time": "19:00",
  "party_size": 12,
  "branch": "large_party",
  "approval": {"by": "manager",
               "status": "pending"},
  "deposit_charged": false,
  "booking_id": null,
  "step": 5,
  "questions_asked": 2
}
```

In context, every LLM call

System prompt with opening hours and booking rules, the guest's latest message, and the last 3 scratchpad steps.

Persists across chats

Seating preference per phone number (for example "window seat"). Nothing else.

Never in context

Other guests' bookings or phone numbers, and payment card details.

Resume rule

On restart, read the scratchpad. If deposit_charged is true, never charge again; if booking_id is set, only confirm.

PART 3 · CONTROLS

Six Controls, Each With an Exact Setting

Control	Where it fires	Setting	Why
Permission scope	Agent start	5 tools; refund_deposit not exposed	Refunds are a manager decision
Branching	After availability	⩽ 8 book; > 8 ask manager; none free: offer 2 nearest times; unclear: ask	Every outcome has a defined path
Retry + breaker	check_availability	3 tries, 1-2-4 s; breaker opens after 5 fails	Booking system is flaky at peak hours
HITL gate	Party > 8 (deposit)	Manager approves or rejects; reject offers a smaller table	Money and kitchen capacity
Time lane	Manager approval	10 min, then "staff will call you back"	Never charge on a timeout
Stops	Whole chat	10 steps, 3 questions	No endless loops or interrogations

Deliberately left out: two-factor judgment. The only money action already has a human gate, so a counter-model would add latency without reducing risk.

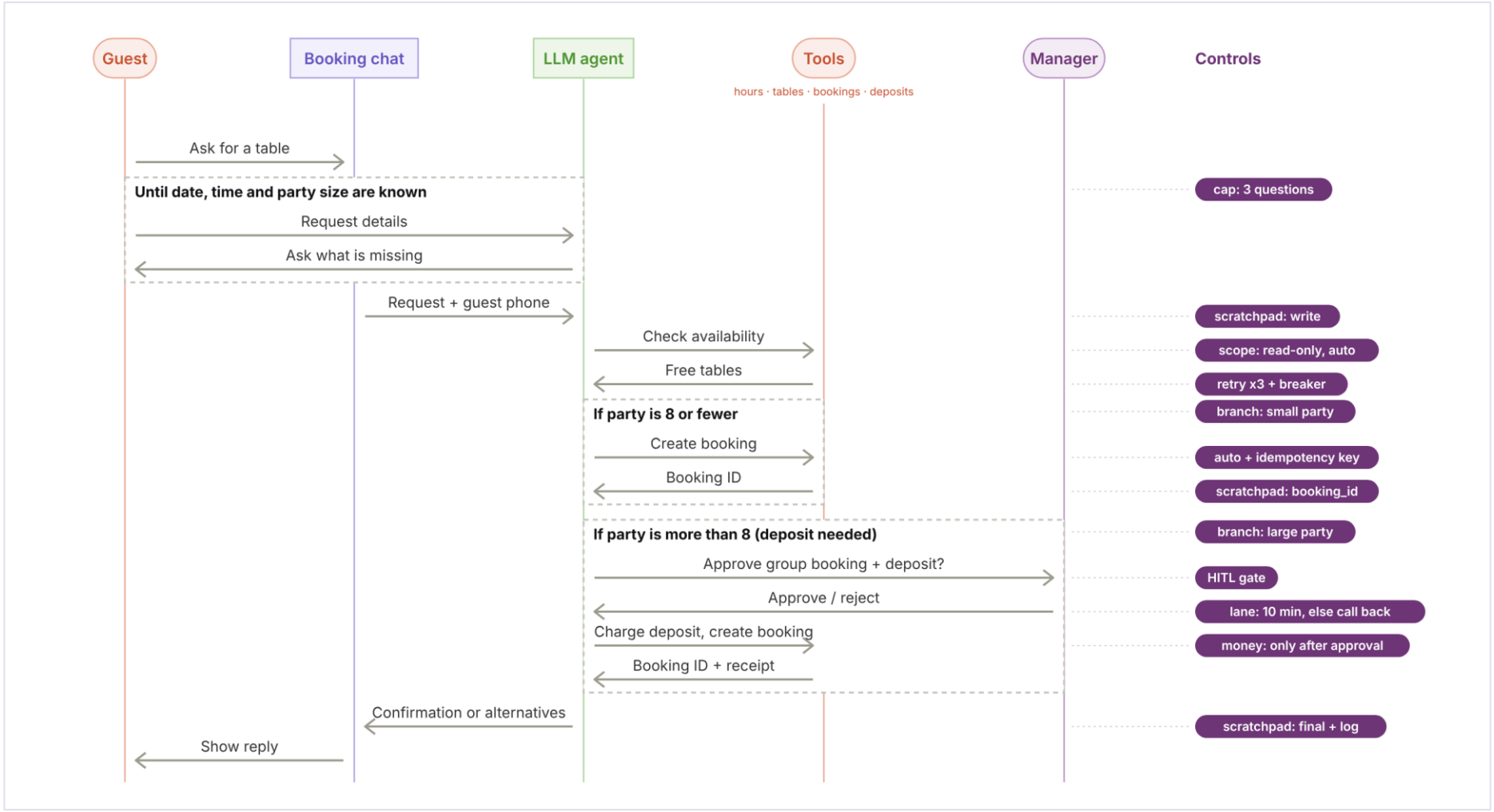
Five Tools, Rated Before They Are Exposed

Tool	Effect	Reversible?	Autonomy
`get_opening_hours`	read	yes	auto
`check_availability`	read	yes	auto, with retry + breaker
`create_booking`	write	yes (can cancel)	auto for ⩽ 8, idempotency key
`cancel_booking`	write	mostly (table may be taken)	auto, guest's own booking only (phone must match)
`charge_deposit`	money	no	only after manager approval
`refund_deposit`	money	no	not exposed: manager only

Rule of thumb: reversibility and blast radius set the autonomy level. Reads run alone, writes run with a key, money waits for a person, and some tools are never exposed at all.

PART 1 · SEQUENCE DIAGRAM

One Booking, End to End



YOUR TURN

Now Design the Customer Support Agent

1. **Sequence diagram**
   Every actor, every loop and condition, and each control tagged where it fires.

2. **Memory and context**
   Scratchpad schema, what each call sees, what persists, what never enters, and the resume rule.

3. **Controls**
   Each control with where it fires, an exact setting and a reason, plus what you left out and why.

4. **Tools**
   Every supportkit tool rated by effect and reversibility, with an autonomy level or "not exposed".

Harder than the example

Real refunds above and below a limit

Tickets that try prompt injection

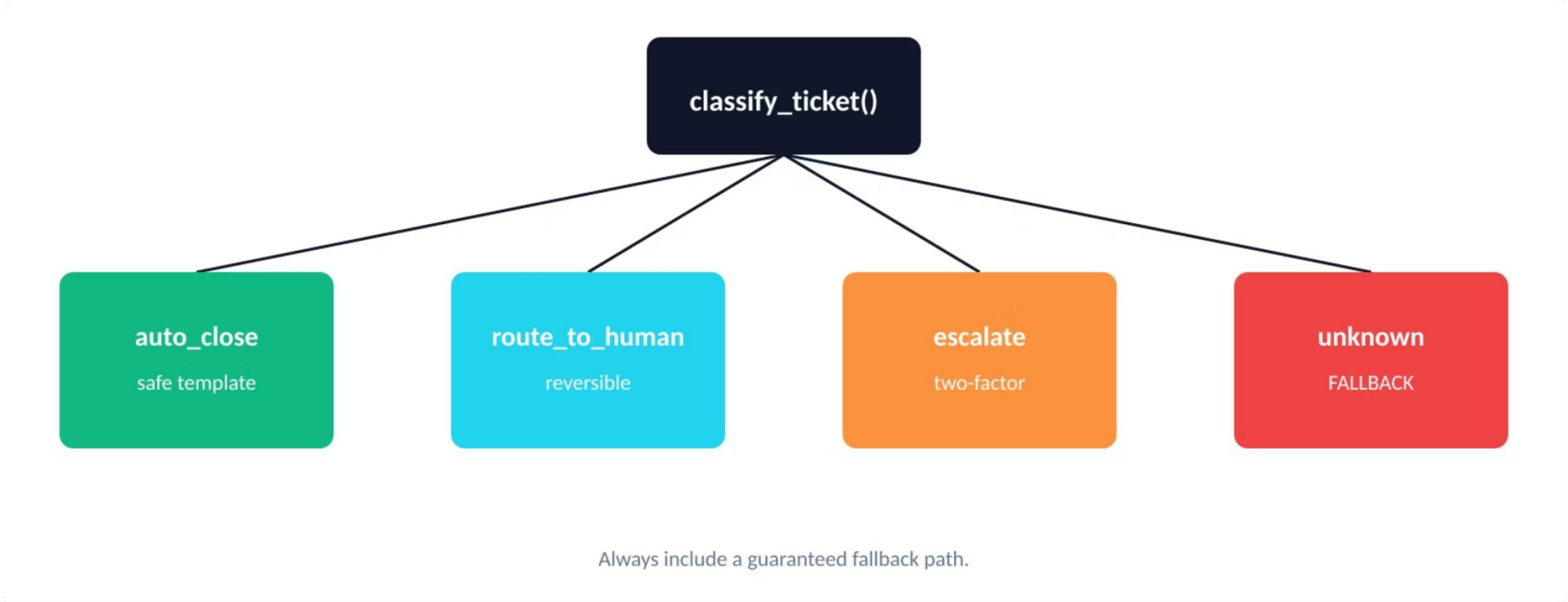
Account changes the agent must not make

Legal threats that must escalate

Test against all 12 supportkit tickets.

STEP 2 · BRANCHES

Define Every Branch, Including Unknown



Write the condition for each branch, not just its name. auto_close only if confidence is high AND the issue class is on the safe list; route_to_human for reversible but ambiguous cases; escalate for high-stakes cases, through two-factor judgment; unknown catches everything else.

STEP 3 · CONTROLS

Pick Controls Per Action, Not Per Agent

Permissions

Which tools exist at all? Allowlist with default deny. Read-only tools can run unattended.

Gates

Which actions need HITL, a counter-model, or both? What happens on reject?

Failure Handling

Retry transient errors only, a circuit breaker per dependency, and a safe default when a time lane expires.

Stops and Budgets

Step cap, question cap, cost cap. What does the agent tell the customer when it stops?

STEP 4 · SCRATCHPAD AND MEMORY

Decide What the Agent Writes Down

Working scratchpad

Per conversation: customer ID, issue class, evidence, proposed action, approvals, step count. Written after every step.

Episodic memory

Across conversations: past cases for this customer, and Reflexion-style critiques of runs that went wrong.

Procedural (skills)

How-to as SKILL.md: refund policy, tone guide, escalation playbook. Loaded only when needed.

For each field, decide when it is written, who reads it, and when it is deleted. If the process crashes right after a refund succeeds, what lets you resume without refunding twice?

Source: LangChain, Context Engineering for Agents